

Lezione 07: Incapsulamento, Costruttori Multipli e Asserzioni

Questo notebook raccoglie tutto il codice della Lezione 07:

- SimpleDate/src/Date.java
- SimpleDate/src/MainDate.java
- SimpleDate/src/TestDate.java

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/  
└─ codice-commentato/  
    └─ Lezione07/  
        └─ SimpleDate  
            └─ src  
                ├── Date.java  
                ├── MainDate.java  
                └─ TestDate.java
```

Argomenti trattati:

- Modificatori di accesso e information hiding: campi privati (`private int day, month, year;`)
- Metodi getter (`getDay()` , `getMonth()` , `getYear()` , `getLang()`) e setter (`setDay(...)` , `setMonth(...)` , `setYear(...)` , `setLang(...)`)
- Costruttori multipli: standard a 3 argomenti, a 2 argomenti con valore di default (tramite `this(...)`) e costruttore di copia
- Internazionalizzazione dei formati di visualizzazione: formato italiano `dd/mm/yyyy` vs statunitense `mm/dd/yyyy`
- Scrittura di unit test basilari con asserzioni (`assert`) e flag JVM `-ea`)

1. Classe `Date` con incapsulamento completo

```
In [1]: class Date {  
    private int day;  
    private int month;  
    private int year;  
    static final String FORMAT_IT = "dd/mm/yyyy";  
    static final String FORMAT_US = "mm/dd/yyyy";  
    private byte lang; // 0: IT, 1: US  
  
    // constructors  
    public Date(int day, int month, int year) {  
        this.day = day;  
        this.month = month;  
        this.year = year;  
        verify();  
        lang = 0;  
    }  
    public Date(int day, int month) {  
        this(day, month, 2025); // default year  
    }  
    public Date(Date other) { // copy constructor  
        this.day = other.day;  
        this.month = other.month;  
        this.year = other.year;  
        verify();  
        lang = other.lang;  
    }  
  
    void verify() {  
        if (year < 0 || month < 1 || month > 12) // year and month check  
            System.out.println("Illegal date!"); // Illegal date!
```

```

        else
            if (day < 1 || day > daysPerMonth(month)) // day check
                System.out.println("Illegal date!"); // Illegal date!
    }

    // getters
    public int getDay() { return day; }
    public int getMonth() { return month; }
    public int getYear() { return year; }
    public byte getLang() { return lang; }

    // setters
    public void setDay(int day) {
        this.day = day;
        verify();
    }
    public void setMonth(int month) {
        this.month = month;
        verify();
    }
    public void setYear(int year) {
        this.year = year;
        verify();
    }
    public void setLang(byte lang) {
        if (lang == 1) this.lang = 1;
        else this.lang = 0;
    }

    // methods
    String printFormat() {
        return lang == 0 ? FORMAT_IT : FORMAT_US;
    }

    String print() {
        return (lang == 0 ? day + "/" + month : month + "/" + day) + "/" + year;
    }

    public String toString() {
        return print();
    }

    static int daysPerMonth(int month) {
        int days;
        switch(month) {
            case 4:
            case 6:
            case 9:
            case 11:
                days = 30;
                break;
            case 2:
                days = 28;
                break;
            default:
                days = 31;
                break;
        }
        return days;
    }
}

```

2. Classe `TestDate` ed Esecuzione

```

In [2]: class TestDate { // test class
        // test setup
        private static Date date = new Date(1, 1, 1970);

        private static void printFormatTest() { // test method
            date.setLang((byte) 1);
            assert date.printFormat().equals(Date.FORMAT_US);
            System.out.println("printFormatTest: OK (" + date.printFormat() + ")"); // printFormatTest: OK
            (mm/dd/yyyy)
        }
    }

```

```

    }

    // run with -ea JVM option, cli: java -ea TestDate
    public static void main(String[] args) {
        printFormatTest(); // execute test
    }
}
TestDate.main(null);

```

printFormatTest: OK (mm/dd/yyyy)

3. Classe `MainDate` ed Esecuzione

In [3]:

```

class MainDate {
    public static void main(String[] args) {
        Date d1 = new Date(-7, 10, 2025); // verify() stampa: Illegal date!
        Date d2 = new Date(20, 10);
        Date d3 = new Date(d2);
        //d3.day = -7; // compile-time error
        System.out.println(d1.toString()); // -7/10/2025
        System.out.println(d2.toString()); // 20/10/2025
        System.out.println(d3.toString()); // 20/10/2025
        System.out.println(d3.getDay()); // 20
        d3.setDay(-6); // illegal date -> verify() stampa: Illegal date!
        d3.setDay(2); // ok
        //Date d4 = new Date();
        //System.out.println(d4.toString());
        d2.setLang((byte) 1); // set format to US
        System.out.println(d2.toString() + " " + d2.printFormat()); // 10/20/2025 mm/dd/yyyy
        System.out.println(d3.toString() + " " + d3.printFormat()); // 2/10/2025 dd/mm/yyyy
    }
}
MainDate.main(null);

```

```

Illegal date!
-7/10/2025
20/10/2025
20/10/2025
20
Illegal date!
10/20/2025 mm/dd/yyyy
2/10/2025 dd/mm/yyyy

```